

TECHNIZER INDIA · TRAINING TRACK

CLAUDE CODE

Complete Guide



What We'll Cover

01

What Is Claude Code?

Agentic harness · CLI architecture · model routing

02

Models & Routing Flows

Sonnet · Opus · Haiku · Ollama local inference

03

Session Management

Named sessions · resume · fork · export · context

04

Slash Commands

Built-in commands · agentic workflows · shortcuts

05

Project Setup & Skills

CLAUDE.md · settings.json · custom skills

	Claude AI	Claude Code	Claude Cowork	Claude Design
For	Everyone	Developers	Non-developers	Designers / PMs
Where	· Browser · Mobile · Desktop (Chat tab)	· Terminal CLI · Desktop (Code tab) · IDE extensions	Desktop (Cowork tab)	Browser (claude.ai/design)
Does	Chat & assist	Writes & runs code	Automates tasks & files	Creates visuals
Input	Conversation	Prompts + code	Tasks & workflows	Prompts + uploads
Output	Text / analysis	Working code	Automated actions	Prototypes / decks
Access	All plans	Pro, Max, Team, Enterprise	Beta	Research preview

Product	Status	Access
Claude AI (chat)	✅ GA	Free + all paid plans
Claude Code (CLI + desktop)	✅ GA	Pro, Max, Team, Enterprise
Claude Cowork	✅ GA (with beta features inside)	Pro, Max, Team, Enterprise
Claude Design	🔬 Research Preview	Pro, Max, Team, Enterprise — rolling out gradually
Claude for Excel / PowerPoint / Word	✅ GA	All paid plans
Claude for Outlook	🧪 Public Beta	All paid plans
Claude Security	🧪 Public Beta	Enterprise only
Claude Mythos Preview	🔒 Gated Research Preview	Invitation-only - Project Glasswing partners

What Is Claude Code?

An agentic harness — not just a model wrapper



Key Insight

Claude Code is a CLI tool (TypeScript / Node) that wraps an agentic loop around the Anthropic /v1/messages API.

The model handles reasoning — Claude Code handles everything else.

Agentic Loop

QueryEngine — repeats until task complete

Tool System

50+ tools: Read, Write, Bash, Git...

Permission Pipeline

Allow / Ask / Deny per tool

Context Manager

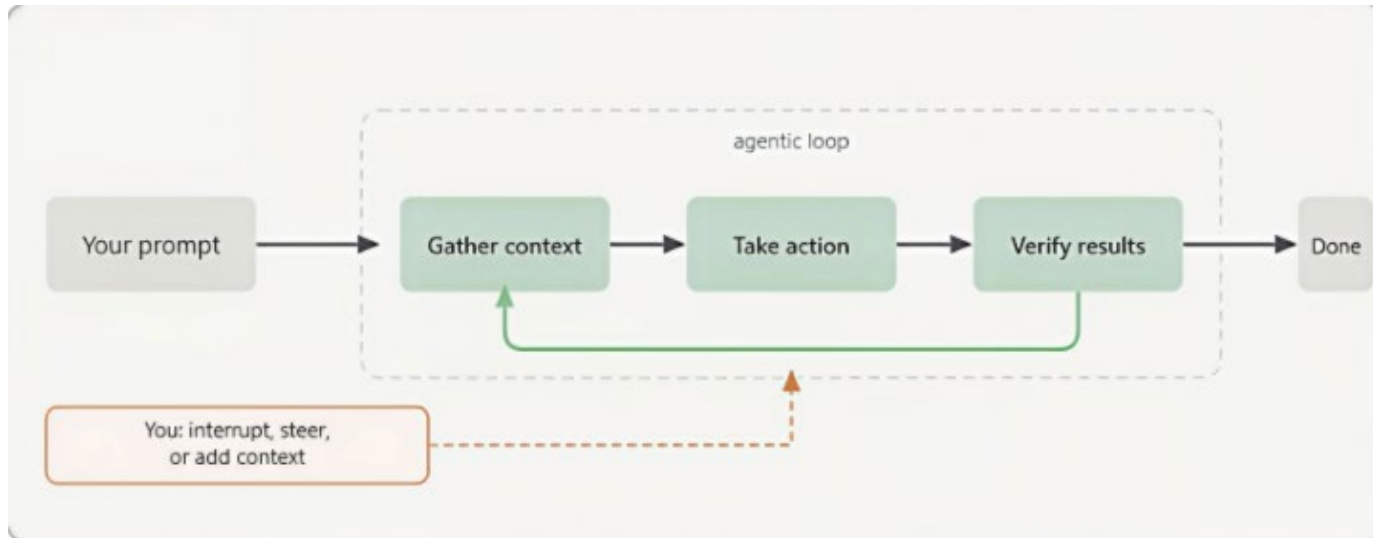
Compaction, pruning, context tracking

Session Persistence

Auto-saves all sessions as .jsonl

The agentic loop

The agentic loop is powered by two components: **models** that reason and **tools** that act. Claude Code serves as the **agentic harness** around Claude: it provides the tools, context management, and execution environment that turn a language model into a capable coding agent.



Claude Code — Runtime Architecture

From terminal command to model response, with tools and the agentic loop

```
YOUR TERMINAL  
$ claude --model claude-sonnet-4-6
```

CLI Parser & Bootstrap

Commander.js — argument parsing, subcommands, flags

Auth — OS keychain, MDM policies, env vars

Resolves model, base URL, working directory, MCP config

QueryEngine

main.tsx

← the agentic loop →

plan → call tool → observe → call API → repeat until done

Tool results feed back
into the loop as new context

tool calls

API calls

Tools (50+ ops)

Read · Write · Edit
Bash · Glob · Grep
Git · WebFetch · MCP

file system · shell · network · integrations

ANTHROPIC_BASE_URL

env var · routes every model request

api.anthropic.com

(default)

Opus · Sonnet · Haiku

cloud-hosted Claude models

localhost:11434

(if BASE_URL overridden)

Ollama → Qwen / local

on-device inference

Inside Claude Code — Pipeline & Bootstrap

From CLI command to ready-to-run agent



Entry & Bootstrap — covered on this slide

01

Entry Points

Routes to execution mode

- CLI
- MCP server
- SDK
- Daemon

Each mode has its own entry handler

02

Bootstrap

Loads runtime essentials

- Configuration
- Telemetry
- Auth credentials
- Networking

Layered: env vars · files · MDM

03

Setup

Prepares the environment

- Working directory
- Hooks
- Plugins
- File watchers

Ready before the first prompt

04

Parallel Prefetch

Fired before heavy imports

- MDM settings
- Keychain
- API preconnect

Cuts perceived startup latency

Parallel side-effects fire on launch — BEFORE heavy module imports —
so the CLI feels instant instead of waiting on the JS bundle.

QueryEngine & the Agentic Loop

The heart of execution — and why it's deliberately simple

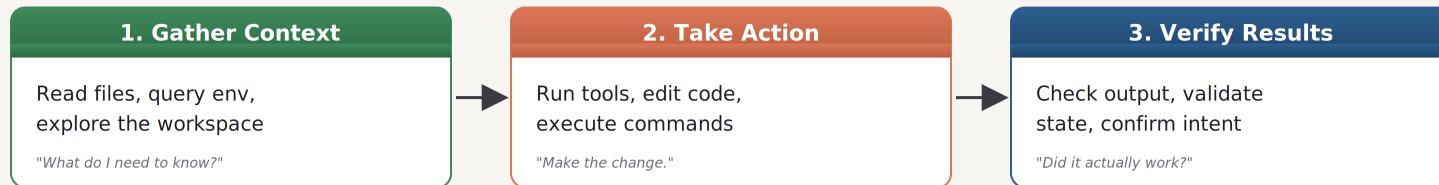


QueryEngine — the heart

Sits between the UI and the Claude API

send → respond → execute tools → return results → repeat

Three phases — they blend, and Claude course-corrects from each step



No classifiers · No RAG · No DAG

just `while(tool_call)` — the model itself decides every step

The Two Routing Flows

Flow 1 — Anthropic Cloud

- 1 Terminal prompt
- 2 Claude Code CLI
- 3 POST `api.anthropic.com/v1/messages`
- 4 Opus / Sonnet / Haiku (cloud)
- 5 Stream response + `tool_use`
- 6 Execute tools locally
- 7 Repeat until done

Flow 2 — Ollama Local

- 1 Terminal prompt
- 2 Claude Code CLI
- 3 POST `localhost:11434/v1/messages`
- 4 Ollama (translates Anthropic format)
- 5 Local inference — GPU/CPU
- 6 Execute tools locally
- 7 Repeat until done

Cloud vs Local — Side by Side

	Flow 1 — Anthropic Cloud	Flow 2 — Ollama Local
Model Quality	Best (Opus/Sonnet/Haiku)	Good (depends on local model)
Cost	API billed per token	Free — electricity only
Privacy	Code sent to Anthropic	Fully local — never leaves machine
Speed	Fast (cloud inference)	Slower — depends on hardware
Tool Calling	Full agentic capability	Depends on local model support
Internet	Required	No — after initial model pull
Setup	Just an API key	Ollama + model download (≥16 GB RAM)

Models at a Glance

Sonnet 4.6

Daily Driver — Default

- ▶ Writing new features & components
- ▶ Fixing standard bugs & refactoring
- ▶ Tests, docs, PR reviews
- ▶ Computer use & vision tasks

Haiku 4.5

The Sprinter — Fast & Cheap

- ▶ Bulk find-and-replace operations
- ▶ Simple lookups & quick answers
- ▶ Boilerplate scaffolding & linting
- ▶ Rate-limit conservation tasks

Opus 4.7

Deep Thinker — Complex Work

- ▶ Hard intermittent / race condition bugs
- ▶ Architecture & system design
- ▶ Security audits & vulnerability analysis
- ▶ 1M context — entire repositories

opusplan

Intelligent Hybrid Mode

- ▶ Plan with Opus — execute with Sonnet
- ▶ Large feature implementations
- ▶ Complex multi-file migrations
- ▶ Best of quality + speed

Switching Models & Effort Levels

How to Switch

Mid-session

```
/model sonnet | /model opus | /model haiku
```

At startup

```
claude --model opus
```

Env var

```
export ANTHROPIC_MODEL="sonnet"
```

Settings file

```
{ "model": "sonnet" } → .claude/settings.json
```

One-shot

```
claude --model haiku -p "Update headers"
```

Effort Levels

/effort low

Fast, less thorough

Yes

/effort medium

Default for Pro/Max

Yes

/effort high

Deep reasoning

Yes

/effort max

Maximum — Opus 4.6 only

Session only

/effort auto

Reset to model default

—

💡 **ultrathink** — add anywhere in a prompt to trigger high effort for that single turn (Claude Code terminal only)

When to Switch — Decision Guide

Stay on Sonnet when:

- ✓ Writing features, fixing standard bugs
- ✓ Refactoring, tests, documentation
- ✓ PR creation and code reviews
- ✓ 90%+ of everyday coding tasks

Stay on Opus when:

- ✓ Hard/intermittent bugs Sonnet can't solve
- ✓ Architecture decisions & design patterns
- ✓ Security audits, unfamiliar codebases
- ✓ Tried Sonnet twice — still off-track

Stay on Haiku when:

- ✓ Bulk boilerplate / copyright header updates
- ✓ Simple find-and-replace across files
- ✓ Quick factual lookups
- ✓ Rate limit conservation

Stay on opusplan when:

- ✓ Large features needing a thorough plan
- ✓ Complex multi-file migrations
- ✓ Want Opus intelligence + Sonnet speed
- ✓ Architecture first, code second

Session Management

All sessions auto-saved locally to `~/.claude/projects/<encoded-path>/*.jsonl` — never auto-deleted. Everything restored on resume: message history, tool results, model, working directory.

Start & Name

```
claude -n "feature-auth"
```

Start named session

```
/rename payment-v2
```

Rename current session

```
/rename
```

Auto-name from history

```
claude -n "bugfix-webhook"
```

Naming convention tip

Resume

```
claude -c
```

Continue last session

```
claude -r
```

Open session picker

```
claude -r "feature-auth"
```

Resume by name

```
/resume
```

Switch mid-session

Context & Export

```
/context
```

Check context usage

```
/compact
```

Summarise older msgs

```
/export payments.md
```

Export as plain text

```
/branch or /fork
```

Fork current session

Project Setup — CLAUDE.md & Settings



CLAUDE.md

- ▶ Persistent memory for Claude — loaded on every session
- ▶ Define tech stack, architecture, coding standards
- ▶ Set rules: What Claude must never do
- ▶ Specify PR and Git standards
- ▶ Created with `/init` command
- ▶ Supports global (`~/ .claude/CLAUDE.md`) and per-project versions



settings.json

```
{
  "model": "sonnet",
  "env": {
    "ANTHROPIC_BASE_URL":
      "http://localhost:11434",
    "ANTHROPIC_AUTH_TOKEN": "ollama",
    "ANTHROPIC_API_KEY": ""
  }
}
```



Folder Structure

<code>.claude/</code>	
<code> settings.json</code>	← model, env vars
<code> commands/</code>	← custom slash cmds
<code> skills/</code>	← reusable skill files
<code> CLAUDE.md</code>	← project memory

Key Shortcuts & Keyboard Controls

Keyboard Shortcuts

Shift + Tab

Cycle permission modes: Normal → Auto-Accept → Plan Mode

Ctrl + O

Toggle verbose — shows extended thinking as gray italic text

Ctrl + G

Open current plan in your default text editor

Ctrl + T

Toggle background task list display

Option/Alt + T

Enable / disable extended thinking

Esc Esc

Time machine — browse all prompts from current session

↑ arrow

Navigate back through past prompts (even from previous sessions)

CLI Flags at Startup

--model <alias>

Set model for this session

--name / -n

Start with a named session

--continue / -c

Resume last session

--resume / -r

Open session picker or resume by name

--from-pr 142

Resume session linked to a PR

--permission-mode plan

Start in Plan Mode

-p "prompt"

Headless / non-interactive mode

The 6 Building Blocks



CLAUDE.md

Project Memory

What Claude knows about your project — always. Loaded every session automatically.



Skills

Auto-Applied Expertise

Claude reads context and invokes the right skill automatically — no prompt needed.



Hooks

Automated Quality Gates

Shell commands that fire automatically around Claude actions — linting, formatting, notifications.



Slash Commands

On-Demand Shortcuts

Reusable prompts you trigger manually with a forward slash. Personal or team-wide.



Agents

Specialist Instances

Isolated Claude instances with their own persona, tools, and context window.



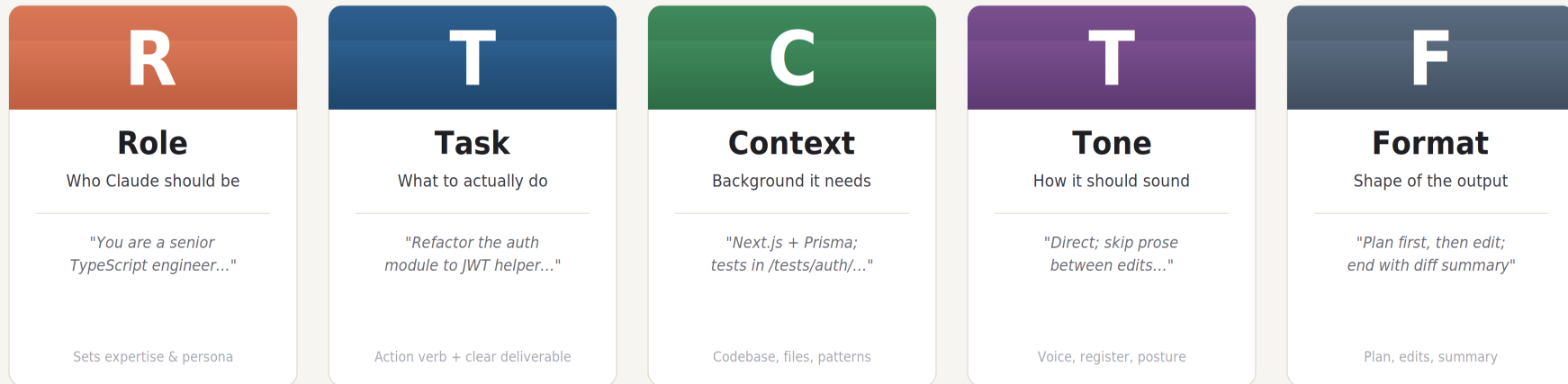
Plugins

Portable Team Distribution

Package all 5 layers above into a single installable unit. One command, full team setup.

The RTCTF Prompt Framework

Five components — for prompts Claude Code can act on



A real RTCTF prompt — refactoring an auth module in Claude Code

- R** You are a senior TypeScript engineer working in this Next.js + Prisma codebase.
- T** Refactor the auth module to use the new JWT helper in lib/auth/jwt.ts.
- C** All callers live in /app/api/auth/* and /pages/login. Follow patterns from lib/auth/jwt.ts.
- T** Be direct — skip prose explanations between edits.
- F** Show a plan first, then edit files. End with diff summary and tests to run.

Five components, five fewer guesses.

In Claude Code's tool-calling loop, every guess compounds into a real action.

RTCTF as a Design Discipline

Authoring config means writing every prompt in advance — for everyone, forever

SURFACE

RTCTF DEMANDED

PLUS

CLAUDE.md

Project's persistent identity

Miss any of the durable four — every per-turn prompt is forced to compensate.



— Task comes per-turn —

Slash Command

.claude/commands/refactor.md

Full RTCTF — the Task is parameterized; everything else is locked in by you.



+ ARGUMENTS

Skill

skills/<name>/SKILL.md

Frontmatter description is the load-trigger — bad description = wrong loads, or no loads.



+ TRIGGER METADATA

Subagent

.claude/agents/<role>.md

Self-contained RTCTF + which tools the agent may call (Format by exclusion).



+ TOOL SCOPE

Authoring is prompt engineering with permanence.

What you bake into config fires on every invocation — make sure it's the right thing.

Built-in Slash Commands

Session & Context

`/clear` `/compact` `/context`

`/rewind` `/rename` `/branch`

`/export` `/copy` `/resume`

Model & Config

`/model` `/effort` `/fast`

`/config` `/theme` `/output-style`

`/status` `/cost` `/usage`

Agentic Workflows

`/batch` `/simplify` `/debug`

`/loop` `/claude-api`

`/plan` `/ultraplan`

Git & Code Review

`/diff` `/security-review`

`/autofix-pr` `/install-github-app`

`/tasks` `/bashes`

Permissions & Tools

`/permissions` `/hooks` `/mcp`

`/add-dir` `/sandbox`

`/memory` `/skills` `/agents`

Utility & Info

`/help` `/doctor` `/stats`

`/insights` `/release-notes`

`/feedback` `/powerup`

Key Takeaways

1

Claude Code is an agentic harness — it executes the loop, tools, and permissions. The model (cloud or local) only handles reasoning.

2

Start with Sonnet. Escalate to Opus only when you've genuinely hit its ceiling. Use Haiku for mechanical, repetitive tasks.

3

Use opusplan for large features: plan with Opus quality, execute with Sonnet speed.

4

Name sessions early with `-n`. One session per task. Use `/compact` before context fills up. Export before big refactors.

5

Slash commands are your control panel: `/model`, `/effort`, `/batch`, `/compact`, `/export`, `/diff` — learn these first.

6

CLAUDE.md + `.claude/settings.json` are your project memory. Set them up once, gain consistent AI behaviour across all sessions.

Agents — Specialist Claude Instances

A Skill tells the main Claude how to handle a task. An Agent is a separate isolated Claude you delegate work to — its own persona, tools, and context window.

`.claude/agents/db-specialist.md`

Allowed: Read · Bash(npx prisma *) · Bash(psql *)

Senior PostgreSQL & Prisma ORM engineer. Thinks exclusively about data — schema, query performance, index strategy, migrations. Does not write application logic.

Focus Areas

- ▶ Review Prisma schema for missing indexes
- ▶ Flag N+1 patterns in Prisma calls
- ▶ Write & validate migration files
- ▶ Suggest query rewrites for performance

`.claude/agents/security-analyst.md`

Allowed: Read · Bash(npm audit) · Bash(grep *)

Senior application security engineer. Reads code exclusively through a security lens. Finds vulnerabilities and explains how to fix them. Tags every finding with OWASP category.

Focus Areas

- ▶ Authentication bypass & broken access control
- ▶ Injection vulnerabilities (SQL, command, path)
- ▶ Sensitive data exposure in logs & responses
- ▶ Insecure token handling & missing validation

Invoke with: "Ask the db-specialist to..." | "Use the security-analyst agent to audit..." | `/agents` (picker)

Agentic Workflow Commands

Bundled Skills — Claude Code v2.1.x

`/batch <instruction>`

Orchestrate large-scale changes across a codebase in parallel. Decomposes into 5–30 independent units, spawns one background agent per unit in isolated git worktrees, each opens a PR.

→ `/batch migrate src/ from Solid to React`

`/simplify [focus]`

Spawns three parallel review agents to check recently changed files for reuse, quality, and efficiency — then applies the fixes automatically.

→ `/simplify focus on memory efficiency`

`/loop [interval] [prompt]`

Run a prompt repeatedly while the session stays open. Claude self-paces if no interval is given. Great for polling or watching long-running processes.

→ `/loop 5m check if the deploy finished`

`/autofix-pr [prompt]`

Spawns a web session watching the current PR for CI failures and reviewer comments, then pushes fixes automatically. Requires the gh CLI.

→ `/autofix-pr fix lint errors in the auth module`

`/ultraplan <prompt>`

Draft a plan in an ultraplan session, review it in the browser, then execute remotely or send back to terminal. Combines deep planning with remote execution.

→ `/ultraplan redesign the payment service`

Commands vs Skills vs Agents

Decision framework — which to use when

Slash Command

Trigger: You type `/command`

Use when you want a repeatable saved prompt triggered manually

Scope: Per-project (`.claude/commands/`) or Personal (`~/.claude/commands/`)

Examples

→ `/commit` `/security-review` `/fix-issue` `/standup`

Skill

Trigger: Claude reads context and invokes automatically

Use when you want Claude to apply expertise based on what it sees — without being asked

Scope: Per-project (`.claude/skills/`) or via Plugin

Examples

→ `code-review` skill fires when you say 'check the code'

→ `write-tests` skill fires when new code is produced

Agent

Trigger: You delegate: `'use the X agent to...'`

Use when you want full isolation — its own persona, tool set, context — for a focused specialist task

Scope: Per-project (`.claude/agents/`) or via Plugin

Examples

→ `db-specialist` reviews the Prisma schema

→ `security-analyst` audits `authService.js`

Hooks — Automated Quality Gates

Shell commands that fire automatically around Claude actions — defined in `.claude/settings.json`

PostToolUse

Fires AFTER Claude writes, edits, or runs something.
Most common use: auto-lint or format every file Claude touches.

PreToolUse

Fires BEFORE Claude runs a command. Can block the action.
Use for approval gates on sensitive operations.

Stop

Fires when Claude finishes a task.
Use for desktop notifications so you can step away during long tasks.

Notification

Fires when Claude needs your attention mid-task.
Route to Slack, email, or desktop notification system.

```
"hooks": {
  "PostToolUse": [{ "matcher": "Write|Edit",    "hooks": [{ "type": "command", "command": "xargs npx eslint --fix" }] }],
  "PostToolUse": [{ "matcher": "Write(*.py)",  "hooks": [{ "type": "command", "command": "xargs python -m black" }] }],
  "Stop":        [{ "hooks": [{ "type": "command", "command": "osascript -e 'display notification \"Claude finished\"'" }] } ]
}
```

Plugins — Package & Share Your Setup

Without plugins: new developers get a Slack message — "copy this folder, edit this JSON, install these servers." Two hours later, everyone has a slightly different setup. Plugins make your entire Claude Code configuration installable in one command.

Plugin Structure

```
my-team-plugin/
├── .claude-plugin/
│   └── plugin.json      ← Manifest ONLY goes here
├── commands/           ← Slash commands
├── agents/              ← Specialist agents
├── skills/              ← Skills
├── hooks/
│   └── hooks.json       ← Hooks config
└── .mcp.json            ← MCP servers
```

plugin.json Manifest

```
{
  "name": "my-team-plugin",
  "version": "1.0.0",
  "description": "Standard Claude Code setup
                 for our engineering team",
  "commands": ["commands/"],
  "agents":    ["agents/"],
  "skills":    ["skills/"],
  "hooks":     "hooks/hooks.json"
}
```

 Clone repo



 Run claude



 /plugin install [url]



 Type /help — done

Four steps. Fifteen minutes. Every developer has the exact same environment.

What is MCP?

and why it changes how AI works with the world

WHAT IS MCP

Model Context Protocol

An open standard by Anthropic (Nov 2024)

“Think of MCP as USB-C for AI —
one universal plug for every tool.”

AI model (Claude) acts as a **MCP Client**

External tools expose themselves as **MCP Servers**

A shared protocol bridges them — **no custom glue code needed**

Claude ↔ MCP Client ↔ MCP Server ↔ Tool

WHY MCP

01 No more custom integrations

Before MCP, every tool needed bespoke glue code per AI app.

02 Secure by design

Servers declare capabilities; clients request with explicit consent.

03 Composable & reusable

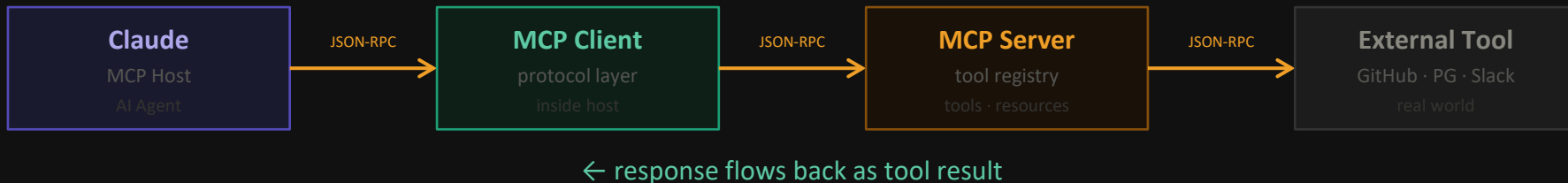
Build one MCP server, plug it into any AI host instantly.

04 Live context, live actions

AI reads real-time data and takes real actions — not just text.

How MCP Communicates

Request → Tool Call → Response — via JSON-RPC over stdio or HTTP/SSE



GitHub MCP

@modelcontextprotocol/server-github

TOOLS

create_issue
list_pull_requests
create_or_update_file
search_repositories

“Create a PR for my bug fix branch”
→ calls create_pull_request

PostgreSQL MCP

@modelcontextprotocol/server-postgres

TOOLS

query (read-only SQL)
list_tables
describe_table
get_schema_info

“Show top 10 users by revenue”
→ calls query → SELECT ...

Slack MCP

@modelcontextprotocol/server-slack

TOOLS

post_message
list_channels
get_channel_history
reply_to_thread

“Post standup summary to #dev”
→ calls post_message



Your MCP Server Code



1. Run locally?



stdio transport

- Claude spawns it as child process
- communicates via stdin/stdout



Claude

stdin/stdout
(stdio)



MCP Server
(Child Process)



2. Deploy to a server (modern)?



HTTP transport

- Claude calls it via HTTP POST
- multiple clients can connect

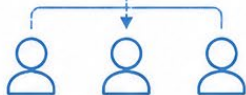


Claude

HTTP
POST



MCP Server
(HTTP)



Multiple Clients



3. Deploy to a server (legacy)?



SSE transport

- Claude connects via persistent stream
- being phased out

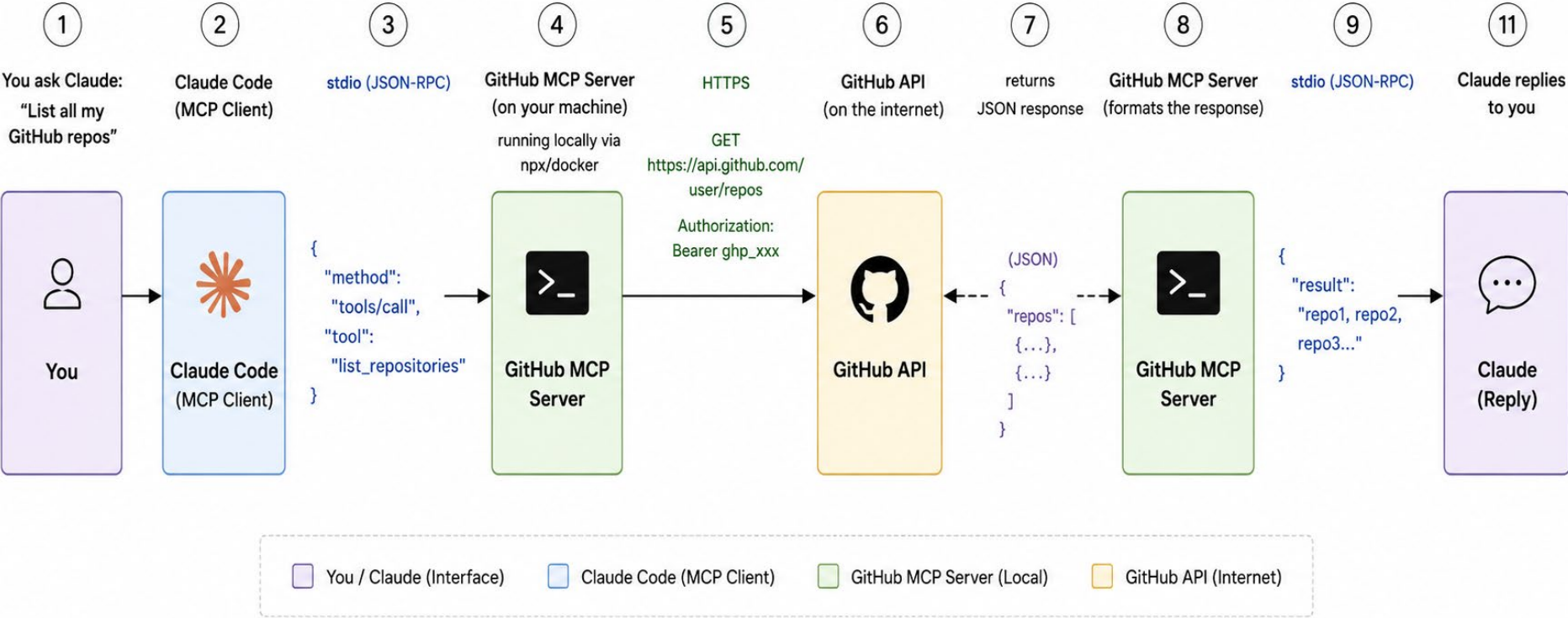


Claude

SSE
(Stream)



MCP Server
(SSE)



Full Bug Lifecycle — End to End with MCP

1

/security-review

Claude audits authService.js

Finds: refresh token store has no expiry cleanup — stale tokens accumulate indefinitely (memory leak + security risk)

2

/create-issue

Claude drafts & posts a structured GitHub issue via MCP

Returns a live GitHub URL. Open it in browser to confirm — full structured issue: summary, behaviour, files, acceptance criteria. No browser tab opened by you.

3

/fix-issue 1

Claude reads issue #1 from GitHub via MCP, locates code, implements fix

Adds cleanupExpiredTokens() to authService.js. Writes a unit test for it. References the issue description to guide the fix.

4

/commit

Claude runs git diff --staged, reads what changed, generates commit message

Produces: fix(auth): remove stale refresh tokens on expiry (closes #1) — reads before it writes.

5

/standup

Claude reads today's git log, sees the commit, drafts standup update

Ready to paste into Slack or Teams. Yesterday / Today / Blockers — factual, 3 bullets max.

Team Rollout — Step by Step

Licensing & Seat Assignment

Assign Standard or Premium seats from the admin panel. Users authenticate with their org account — no individual API keys needed.

Standard = included limits · Premium = higher limits for power users

Create Team Configuration Repository

Dedicated Git repo for Claude Code config: plugin, shared CLAUDE.md templates, approved MCP server list. Separate from product code.

Structure: plugin/ · templates/ · README.md

Write CLAUDE.md Templates Per Project Type

Create templates for each project type (Node.js microservice, Python pipeline, React frontend...). New projects start from the template.

Commit CLAUDE.md to each product repo so every team member gets it

Build & Publish the Plugin

Package commands, skills, agents, and hooks into the plugin. Push to GitHub or internal Git host — this is your team marketplace URL.

/plugin install [url] → instant full setup for any developer

Admin Policy Setup

Push managed settings to all users via admin panel: approved MCP servers, deny list, team marketplace URL. Developers cannot override managed settings.

```
"managedSettings": { "deniedMcpServers": ["*personal*"], "deny": ["Read(/.env*)"] }
```

Developer Onboarding: 4 Steps, 15 Minutes

Install Claude Code → authenticate → clone repo → open project → /plugin install [url] → /help to verify team commands appear.

Every developer has the exact same environment from day one

Q&A Scenarios — Common Questions

Most frequent audience questions and suggested trainer responses

Q: Is our source code sent to Anthropic?

A: By default, yes — to provide responses. Enterprise: zero data retention option via API. The deny list in settings.json blocks Claude from reading secrets or proprietary files.

Q: What if Claude makes a mistake and breaks code?

A: Three safeguards: (1) Claude shows its plan before destructive operations. (2) Always work in a branch — never on main. (3) Every change is tracked via Git — git diff and git checkout bring you back.

Q: We use GitHub Copilot already. Why Claude Code?

A: Different tools. Copilot = inline completion while typing. Claude Code = terminal agent that plans, reasons across files, and executes sequences. Many teams use both — they're complementary.

Q: How much does this cost for 50 developers?

A: Team plan: Standard seats (included limits) + Premium seats (higher limits). extra-usage option with per-user spending caps. Run a pilot with a small group to understand actual consumption first.

Q: Can Claude Code access our internal documentation?

A: Yes — via MCP. If your documentation system has an MCP server, Claude queries it directly, reasoning across code and docs simultaneously. Powerful for onboarding and architecture questions.

Let's Connect with us!



Location

Pune, Maharashtra, India



Phone

+91 9822025942



Website

www.technizerindia.com



Email

info@technizerindia.com

We'd love to connect!

Reach out for collaborations, inquiries, or expert technology consulting.





THANK YOU